

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x06 实验报告

硬盘驱动与文件系统

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. MBR 分区表

首先我们需要根据 MBR 的结构定义，利用 `MbrPartition` 的 `parse` 方法来对四个 `partition` 依次解析，如下

```
for i in 0..4 {
  let offset = 0x1BE + i * 16;
  partitions.push(
      // FIXED: parse the mbr partition from the buffer
      //      - just ignore other fields for mbr
      MbrPartition::parse(buffer[offset..offset + 16].try_into().unwrap()),
  );
```

然后我们需要利用 `define_filed!` 完成内部数据的解析提取，由于 `CHS` 的 `start` 和 `end` 中的 `sector` 和 `cylinder` 不是占据整个字节的，因此要另外写 `start_sector` 等函数进行解析提取，部分代码如下

```
define_field!(u8, 0x00, status); // Status (bit 7 set = active or bootable)
define_field!([u8; 3], 0x01, chs_start); // CHS address of first partition sector
(3 bytes)
define_field!(u8, 0x04, partition_type); // Partition type
define_field!([u8; 3], 0x05, chs_end); // CHS address of last partition sector (3
bytes)
define_field!(u32, 0x08, begin_lba); // LBA of partition start
define_field!(u32, 0x0C, total_lba); // Number of sectors in partition

pub fn is_active(&self) -> bool {
    self.status() == 0x80
}

pub fn begin_head(&self) -> u8 {
    self.chs_start()[0]
}

pub fn begin_sector(&self) -> u8 {
    self.chs_start()[1] & 0x3F
}

pub fn begin_cylinder(&self) -> u16 {
    ((self.chs_start()[1] as u16 & 0xC0) << 2) | self.chs_start()[2] as u16
}

// ..... end_head(), end_sector()... is the same as above,
// see entry.rs for the full code
```

然后我们对这部分进行功能测试，需要在根目录的 `Cargo.toml` 的 `member` 中加入 `"crates/storage",` 表项，然后再 `storage/` 下运行 `cargo test`，可以看到，可以通过对应的测试，说明实现大致无误。

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/storage$ cargo test -- --nocapture
running 1 test
Partition Meta Data {
    Active: true,
    Begin Head: "0x01",
    Begin Sector: "0x0001",
    Begin Cylinder: "0x0000",
    Partition Type: "0x0b",
    End Head: "0xfe",
    End Sector: "0x003f",
    End Cylinder: "0x02fc",
    Begin LBA: "0x0000003f",
    Total LBA: "0x00bb867e",
}
test partition::mbr::entry::tests::partition_test ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00
s

   Doc-tests ysos_storage

running 1 test
test crates/storage/src/common/macros.rs - common::macros::define_field (line 5) ... ignored

test result: ok. 0 passed; 0 failed; 1 ignored; 0 measured; 0 filtered out; finished in 0.00
s

all doctests ran in 0.40s; merged doctests compilation took 0.40s
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/storage$
```

2. ATA 磁盘驱动

我们要实现的是 LBA（Logical Block Address, 逻辑块寻址） 读写命令和 `IdentifyDevice` 这三个命令，为了方便实现，封装了一个发送命令并等待设备就绪的函数 `write_command()`。

2.1. 发送命令

对于 `write_command()`，我们先根据 `block` 的值来解析出 `lba` 地址的低中高八位，然后设置 `drive` 寄存器的值。其中 `drive` 寄存器低四位放在 `block` 高四位，而 `drive` 的高三位需要设为 `1`（第六位表示选择 LBA 寻址方式），第五位为主/从设备的 `drive` 选择位。同时把命令写入 `command` 寄存器。

然后等待设备就绪，判断是否出错，随后等待数据请求就绪（虽然我们这里只实现了读写和 `IdentifyDevice` 三个命令，但由于 `AtaCommand` 里面还有很多别的命令是不需要等待数据请求就绪的，因此这里加了个特判） `FIXME` 部分代码如下

```
// FIXED: store the LBA28 address into four 8-bit registers
//      - read the documentation for more information
//      - enable LBA28 mode by setting the drive register
self.lba_low.write(bytes[0]);
self.lba_mid.write(bytes[1]);
self.lba_high.write(bytes[2]);

self.drive.write(0xE0 | (drive << 4) | (bytes[3] & 0x0F));
// FIXED: write the command register (cmd as u8)
self.command.write(cmd as u8);

// FIXED: poll for the status to be not BUSY
self.poll(AtaStatus::BUSY, false);

// FIXED: poll for the status to be not BUSY and DATA_REQUEST_READY
if cmd == AtaCommand::ReadPio
    || cmd == AtaCommand::WritePio
    || cmd == AtaCommand::IdentifyDevice
{
    self.poll(AtaStatus::BUSY, false);
    self.poll(AtaStatus::DATA_REQUEST_READY, true);
}
```

2.2. 磁盘识别

首先我们要实现的是 `identify_drive()` 函数，根据 `FIXME` 中的提示补全即可，具体请见 `ata/bus.rs`。

然后，我们要实现的是 `open` 函数，来获取磁盘信息，我们要获取的部分是 `serial`，`model` 以及 `blocks` 的值，首先前面两个我们可以通过取 `buf` 中的对应段即可（因为 `X86` 架构是小端序，需要用 `to_be_bytes` 来获取正确的端序）。

但是如果我们直接在 `buf` 中对应的段取出 `blocks` 的值，会得到 `120.00009 GiB` 的输出，这是因为解析数值的时候也会按照小端序来解析，我们可以直接根据原始数据 `res` 中的对应两个 `word` 来拼出 `blocks` 正确的值。代码如下

```rust
let serial = {
    /* FIXED: get the serial from buf */
core::str::from_utf8(&buf[ATA_IDENT_SERIAL..ATA_IDENT_SERIAL + 20])
        .unwrap_or("")
        .trim()
        .into()
};
let model = {
    /* FIXED: get the model from buf */
core::str::from_utf8(&buf[ATA_IDENT_MODEL..ATA_IDENT_MODEL + 40])
        .unwrap_or("")
        .trim()
        .into()
};
let blocks = {
    /* FIXED: get the block count from buf */
    ((res[61] as u32) << 16) | (res[60] as u32)
};
```

然后我们再 `lib.rs` 中加入 `AtaDrive::open(0, 0)`，然后 `make run` 一下，可以看到磁盘信息正确读取了！

```
[INFO] ysos_kernel::drivers::ata - Initialized ATA Buses.
[INFO] ysos_kernel::drivers::ata::bus - Identifying drive 0
[INFO] ysos_kernel::drivers::ata - Drive QEMU HARDDISK QM00001 (504 MiB) opened
[+] App list:
  App Name        | Size       | Entry Point
  ----------------+------------+--------------------
  counter         |    38992 B | 0x0000111100001470
  mq              |    38904 B | 0x00001111000017e0
  fork_test       |    35456 B | 0x00001111000011d0
  sleep           |    34416 B | 0x00001111000010a0
  hello           |    34320 B | 0x0000111100001020
  dinner          |    34480 B | 0x00001111000014f0
  fish            |    38792 B | 0x0000111100001320
  factorial       |    34872 B | 0x0000111100001490
  sh              |    35160 B | 0x0000111100001530

Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>]
```

2.3. 读写数据

我们要实现的是 `read_pio` 和 `write_pio` 函数，这里参考 `FIXME` 中的提示进行补全代码即可。这里使用了 `.to_le_bytes()` 函数来把一个 `word` 按照小端序拆成两个字节，以及使用 `from_le_bytes()` 把两个字节按照小端序拼成一个 `word`。`FIXME` 部分代码如下

```rust
// FIXED: read the data from the data port into the buffer
//      - use `buf.chunks_mut(2)`
//      - use `self.read_data()`
//      - ! pay attention to data endianness
for chunk in buf.chunks_mut(2) {
    // use .to_le_bytes() to convert data endianness
    let data = self.read_data().to_le_bytes();
    chunk[0] = data[0];
    if chunk.len() > 1 {
        chunk[1] = data[1];
    }
}


// FIXED: write the data from the buffer into the data port
//      - use `buf.chunks(2)`
//      - use `self.write_data()`
//      - ! pay attention to data endianness
for chunk in buf.chunks(2) {
    let mut data = [0u8; 2];
    data[0] = chunk[0];
    if chunk.len() > 1 {
        data[1] = chunk[1];
    }
    self.write_data(u16::from_le_bytes(data));
}
```

然后要用上面这些实现的命令来实现块设备，`block_count` 直接返回 `blocks` 的值即可；`read_block` 和 `write_block` 需要先获取总线的锁，然后调用对应的 `read_pio()` 或者 `write_pio()` 即可。注意这里 `read_pio` 需要传入一个可变的引用，因为要把值从端口传入缓冲区。这部分代码请见 `ata/mod.rs`。

最后需要实现外层的块设备，这里的 `read_block` 和 `write_block` 只需要根据分区的起始 `LBA` 以及块内偏移计算出物理块地址，然后委托给内层的物理块设备即可，代码请见 `partition/mod.rs`。

为了可以输出首个分区的类型信息，需要在 `partition` 中的 `Debug` trait 中的 `fmt` 加入一个 `type` 作为输出，这个需要在 `partition` 的定义中多增加一个成员 `part_type`，并且在 `new` 的时候加入把这个 `type_partition` 也传入进去。

然后我们把日志等级设置为 `Trace`，`make run` 一下可以看到，我们可以正确的在 `partition` 中得到首个分区的相关信息。

```
[TRACE] ysos_kernel::drivers::ata - Opening drive 0@0...
[INFO] ysos_kernel::drivers::ata - Initialized ATA Buses.
[INFO] ysos_kernel::drivers::ata::bus - Identifying drive 0
[INFO] ysos_kernel::drivers::ata - Drive QEMU HARDDISK QM00001 (504 MiB) opened
[TRACE] ysos_storage::partition::mbr - Partition 0: Partition Meta Data {
    Active: true,
    Begin Head: "0x01",
    Begin Sector: "0x0001",
    Begin Cylinder: "0x0000",
    Partition Type: "0x06",
    End Head: "0x0f",
    End Sector: "0x003f",
    End Cylinder: "0x03ff",
    Begin LBA: "0x0000003f",
    Total LBA: "0x000fbfc1",
}
[INFO] ysos_kernel - Partition 0 Info:Partition {
    type: "0x06",
    offset: 63,
    size: 1032129,
}
```

可以看到，`parse` 得到的第 0 个分区的偏移量与 Begin LBA 对应，大小 size 和类型 type 也和 `MBR` 中的信息一致。这里的 `0x06` 类型指的是 FAT16B 类型，是 FAT16 文件格式的一种。

3. FAT16 文件系统

3.1. BPB

这部分我们要做的是根据文档中对 BPB(Bios Parameter Block) 的结构定义来实现 `Fat16Bpb` 的定义，这部分代码请见 `bpb.rs`，这里不再赘述。实现完之后注释掉 `mod.rs` 中对其他模块的引用之后，运行测试，可以看到测试均可以通过！

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/storage/src/fs/fat16$ cargo test --pack
age ysos_storage bpb::tests
ore: `_buf`

warning: `ysos_storage` (lib test) generated 4 warnings (run `cargo fix --lib -p ysos_st
orage --tests` to apply 4 suggestions)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.19s
     Running unittests src/lib.rs (/home/linkzelda/OS-lab/target/debug/deps/ysos_storage
-a8f099cc3289439a)

running 2 tests
test fs::fat16::bpb::tests::test_fat16_bpb_1 ... ok
test fs::fat16::bpb::tests::test_fat16_bpb_2 ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out; finished in
0.00s
```

3.2. DirEntry

首先我们需要实现 `is_directory` 和 `is_long_name` 函数，这个使用 `bitflag!` 宏配合 `.contains()` 判断文件属性的对应位即可。还需要实现一个 `is_valid` 函数，一个目录项合法需要它同时满足不是文件目录末尾，且该文件未被删除，用 `is_eod` 和 `is_unused` 判断即可。

然后要实现对目录项的解析，按照文档中的结构定义实现即可，这里需要对时间解析为 `FsTime` 类型，因为这里只需要精确到秒，因此不需要解析毫秒位。将 `date` 放在高 16 位，`time` 放在低 `16` 位，然后在 `parse_datetime` 中进行对应的解析即可（这里年份是从 `1980` 年开始的偏移，因此需要加上 `1980` ）

然后需要实现对短文件名的解析，这里根据代码中的注释进行实现即可，主要就是将文件名 `basename` 和扩展名 `ext` 分开，然后检查长度/是否存在非法字符/格式是否正确等，如果出现非法则返回异常，否则返回解析出来的文件名和扩展名。

上述的代码请见 `direntry.rs` ，这里不再赘述。实现完之后按照之前类似的方法，进行测试，可以看到通过了对目录项解析的测试！

```
warning: `ysos_storage` (lib test) generated 4 warnings (run `cargo fix --lib -p ysos_stor
age --tests` to apply 4 suggestions)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.59s
     Running unittests src/lib.rs (/home/linkzelda/OS-lab/target/debug/deps/ysos_storage-a
8f099cc3289439a) …

running 1 test
test fs::fat16::direntry::tests::test_dir_entry ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 3 filtered out; finished in 0.
00s

(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/storage/src/fs/fat16$
```

3.3. Fat16Impl

首先我们要算出根目录占用的扇区个数以及起始扇区号，前者可以直接根据目录项的数目乘以每个目录占用的字节数 32 然后除以扇区大小即可（注意这里需要上取整），后者可以根据保留区大小以及 FAT 的大小计算出来。

然后要实现簇号到扇区号的转换，由于 FAT 的前两个条目是保留的，因此对于簇号为 `c` 的簇的真实簇号为 `c-2` ，这里还需要判断簇号的是否在合法的范围内，然后计算出该簇对应的首个扇区。

此外，我们还实现了若干辅助函数，`next_cluster` 用于提取当前簇的下一个簇，方法是先计算出当前簇在 `FAT` 表中的偏移以及当前 `FAT` 表所在的扇区，然后提取这个位置的值就是下一簇号了（还需要判断一下是否为损坏的簇或者是已到达 `FAT` 表的末尾）

还有 `get_dir_entries` 函数，用于读取给定目录下的所有条目的信息，对于根目录，存储在 `FAT` 区后数据区之前，我们可以直接把这部分所有的扇区读完；对于子目录，可能会横跨多个簇，因此需要把该簇里面所有的扇区读完之后，再通过之前实现的 `next_cluster` 函数去到下一个簇。

最后还实现了一个 `find_entry` 函数，用于查找某个路径下的目录项，我们将路径拆分成文件/文件夹名路径，然后每次再当前目录下用 `get_dir_entries` 取出所有条目来寻找是否存在路径指定的下一个目录项，如果存在则跳转到这个目录之后继续寻找。

接下来，需要实现 `Fat16` 的文件系统，对于 `read_dir`，我们直接调用 `find_entry` 函数即可，得到路径对应的目录后再使用 `get_dir_entries` 获取所有的条目并返回其 `Vec` 的迭代器；对于 `open_file`，也是使用 `find_entry` 得到目录，然后返回该目录的 `FileHandle`；对于 `metadata`，我们直接调用 `find_entry` 和 `.as_meta` 获得 `metadata` 即可，根目录是没有 `metadata` 的，因此我们要手动构造一个；最后的 `exists` 只要用 `find_entry` 看看是否可以取得一个合法目录即可。

上述代码请见 `fs/fat16/impls.rs`。

最后，要为 `File` 实现 `Read` trait，即把一个文件的内容读取到缓冲区中，具体而言，就是从当前文件指针指向的地址开始向后读取，知道缓冲区满或者抵达 `EOF` 为止。我们计算出当前地址偏移对应的簇偏移，扇区偏移以及扇区内偏移然后读取之后复制到缓冲区即可，如果在多个簇内，还需要使用 `next_cluster` 去到下一个簇中，中间需要注意复制的文件字节数不要超过缓冲区大小。

这部分代码请见 `fs/fat16/file.rs`。

4. 接入操作系统

这部分我们需要实现 `ls` 和 `cat` 命令。

4.1. 列出目录

首先我们实现 `ls` 命令，我们要做的就是把当前路径下获取的目录条目信息打印出来，具体的格式详见 `drivers/filesystem.rs`。

然后需要添加对应的系统调用给用户态使用，然后分发给 `sys_list_dir` 函数把路径转化为字符串，然后委托给我们上面实现的 `ls` 函数中即可，具体代码请见 `service.rs` 等。

4.2. 读取文件

我们采用 `Open - Read - Close` 过程，在 `sh` 中就是每次打开一个文件，然后一直用缓冲区进行读取，最后关闭该文件。

`sys_read` 我们先前已经实现了，因此我们只需要在之前的 `resource.rs` 中把新实现的 `FileHandle` 注册到里面即可，把 `read` 操作委托给文件句柄的 `read`。

因此我们只需要实现 `sys_open` 和 `sys_close`， 我们可以调用 `.open_file` 函数得到 `FileHandle`，然后插入到内核的 `ResourceSet` 中即可，关闭文件也是类似的，从 `ResourcesSet` 中删除该句柄即可。由于这里的 `ResourceSet` 是 `pub(super)` 的，因此我们在 `ProcessData` 中加入了 `open` 和 `close` 函数封装了对外的对应操作。

最后我们在 `lib.rs` 中加入文件系统的初始化，并在 `sh` 中加上 `ls` 和 `cat` 指令的操作，调用对应的系统调用即可。

我们可以看到可以通过 `ls` 和 `cat` 指令正确读取出 `eps/hello.txt` 的内容！

```
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] ls
Name                    Size Created
--------------- ---------- -------------------
QEMU VVF.AT            0.0 B 1970-01-01 00:00:00
APP/                       - 2026-05-09 13:17:14
EFI/                       - 2026-03-19 15:40:02
KERNEL.ELF           408.7 K 2026-05-17 20:00:54
NVVARS                10.3 K 2026-05-17 18:44:58
HELLO.TXT             31.0 B 2026-05-17 18:44:58
[>] cat hello.txt
Hello filesystem from 24325005!
```

5. 探索 Linux 文件系统

5.1. procfs

全称为 Process File System，是一个伪文件系统。执行 `ls /proc` 可以看到其下保存了 `cpuinfo` 等信息，里面以数字命名的目录对应着正在运行的进程的 PID，其内保存了该进程运行时的相关信息，如 `/cmdline` 就保存了运行该进程时用的命令行参数

```
(base) linkzelda@LAPTOP-QFAEHS2I:/proc/779$ ls
arch_status  coredump_filter  gid_map            map_files  ns            personality  smaps        task
attr         cpuset           io                 maps       numa_maps     projid_map   smaps_rollup timens_offsets
auxv         cwd              ksm_merging_pages  mem        oom_adj       root         stack        timers
cgroup       environ          ksm_stat           mountinfo  oom_score     sched        stat         timerslack_ns
clear_refs   exe              latency            mounts     oom_score_adj schedstat    statm        uid_map
cmdline      fd               limits             mountstats pagemap       sessionid    status       wchan
comm         fdinfo           loginuid           net        patch_state   setgroups    syscall
(base) linkzelda@LAPTOP-QFAEHS2I:/proc/779$ cat ./cmdline
/home/linkzelda/.vscode-server/extensions/rust-lang.rust-analyzer-0.3.2896/server/rust-analyzer(base) linkzelda@LAPTO
```

`/cpuinfo` 存储了各个 CPU 的硬件信息，如型号，cache 大小，主频等，

```
(base) linkzelda@LAPTOP-QFAEHS2I:/proc$ cat cpuinfo
processor       : 0
vendor_id       : GenuineIntel
cpu family      : 6
model           : 170
model name      : Intel(R) Core(TM) Ultra 9 185H
stepping        : 4
microcode       : 0xffffffff
cpu MHz         : 3072.000
cache size      : 24576 KB
physical id     : 0
```

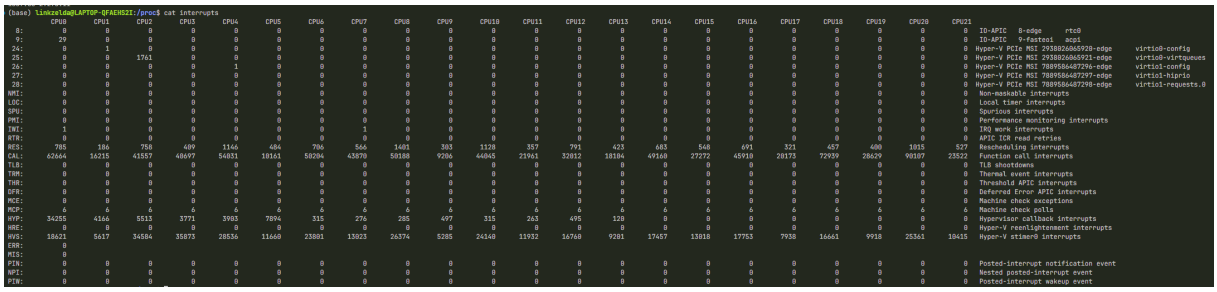
`/meminfo` 则存储了内存相关的信息，如内存总量，空闲的内存量等

```
(base) linkzelda@LAPTOP-QFAEHS2I:/proc$ cat meminfo
MemTotal:       16173732 kB
MemFree:        13659156 kB
MemAvailable:   13709352 kB
Buffers:            2416 kB
Cached:           206512 kB
```

`/loadavg` 存储了近 1, 5, 15 分钟的平均负载（正在使用 CPU 或等待 CPU 的进程数），还有(运行中进程/总进程数)以及最近创建的进程的 PID；而 `/uptime` 则存储了从上次启动到目前的总运行时间，以及各个 CPU 的空闲时间之和

```
(base) linkzelda@LAPTOP-QFAEHS2I:/proc$ cat loadavg
0.11 0.11 0.04 1/441 8535
(base) linkzelda@LAPTOP-QFAEHS2I:/proc$ cat uptime
1337.85 29273.11
```

`/interrupts` 中存储了各个 CPU 各类硬件中断的执行次数，如 `RES` 表示重调度中断，`CAL` 函数调用中断的次数比较多；由于是在 `WSL2` 上运行，因此 `HYP` 和 `HVS` 中断的次数也比较多（前者主要用来处理 I/O 等事件，后者主要用于作为计时器使用）。



`/self` 指向的是执行该命令的进程自己，因此 `cat /proc/self/status` 这条指令会获取 `cat` 命令对应的进程名，进程编号，虚拟内存大小以及占用峰值等。

```
(base) linkzelda@LAPTOP-QFAEHS2I:/proc$ cat self/status
Name:   cat
Umask:  0022
State:  R (running)
Tgid:   13657
Ngid:   0
Pid:    13657
PPid:   655
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 1000
NStgid: 13657
NSpid:  13657
NSpgid: 13657
NSsid:  655
Kthread:        0
VmPeak:     3272 kB
VmSize:     3272 kB
```

`proc/self/smap` 会展示 `cat` 进程的各个段的内存映射信息，如地址范围，权限（可读? 可写? 可执行? 私有映射?），以及映射的虚拟内存大小等。

```
(base) linkzelda@LAPTOP-QFAEHS2I:/proc$ cat ./self/smaps
5ac172a3c000-5ac172a3e000 r--p 00000000 08:30 56447                      /usr/bin
/cat
Size:                  8 kB
KernelPageSize:        4 kB
MMUPageSize:           4 kB
Rss:                   8 kB
Pss:                   8 kB
Pss_Dirty:             0 kB
Shared_Clean:          0 kB
Shared_Dirty:          0 kB
```

`/proc/sys/net/ipv4/ip_forward` 是一个接口，向里面写入 `1` 实际上是将对应的变量改为了 `1`，启动了内核的 IPv4 数据包转发功能。因此我们在 Linux 这样一个“一切皆文件”的系统中想要修改一个运行的参数时，不需要专门实现一个新的系统调用或者相关的修改脚本，可以使用命令行工具（内部采用 `open` 等系统调用实现），通过 Linux 的伪文件系统的接口来进行修改，大大简化的相关的操作！

5.2. devfs

全称为 Device File System，设备文件系统，也是一个为文件系统。

`/null` 读取会直接返回 EOF，向其中写入的所有数据都会被丢弃，常用来屏蔽程序的输出结果；`/zero` 会读取到无尽的 `0x00` 字节字符，常用于创建巨大且内容为空的磁盘镜像或者擦除磁盘的所有内容；`random` 会产生硬件级随机数，会从硬件噪声、用户输入等随机事件中收集“熵”来生成随机数（“熵”不足是会阻塞）；而 `urandom` 时伪随机数生成器，不会阻塞。

`/kmsg` 可以用来访问内核消息的缓冲区，用户空间的进程可以通过此接口直接读写内核日志记录，便于用户程序进行调试。如下图，可以看到缓冲区中包含了内核启动时输出的系统启动的相关信息。

```
(base) linkzelda@LAPTOP-QFAEHS2I:/dev$ head kmsg
5,0,0,-;Linux version 6.6.87.2-microsoft-standard-WSL2 (root@439a258ad544) (gcc (GCC) 11.2.0, GNU ld (GNU Binutils) 2.37) #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025
6,1,0,-;Command line: initrd=\x5cinitrd.img WSL_ROOT_INIT=1 panic=-1 nr_cpus=22 hv_utils.timesync_implicit=1 console=hvc0 debug pty.legacy_count=0 WSL_ENABLE_CRASH_DUMP=1
6,2,0,-;KERNEL supported cpus:
6,3,0,-;  Intel GenuineIntel
6,4,0,-;  AMD AuthenticAMD
6,5,0,-;BIOS-provided physical RAM map:
6,6,0,-;BIOS-e820: [mem 0x0000000000000000-0x000000000009ffff] usable
6,7,0,-;BIOS-e820: [mem 0x00000000000e0000-0x00000000000e0fff] reserved
6,8,0,-;BIOS-e820: [mem 0x0000000000100000-0x00000000001fffff] ACPI data
6,9,0,-;BIOS-e820: [mem 0x0000000000200000-0x00000000f7ffffff] usable
```

WSL2 的 `/dev` 下有 `/sda`，`/sdb` 等文件，这些代表着物理硬盘，直接读写相当于是直接在磁盘的扇区上进行读写。而 `/sda1` 等指的是对应的 `/sda` 等物理硬盘的第一个分区。

`/ttyX` 代表终端设备，提供文本交互的接口；`/loopX` 代表回环设备，将普通文件虚拟为块设备，用于挂载镜像等；`/srX` 代表光盘驱动器。

有如下图的四种软连接，这样设计的好处是路径不会变化，采用 `/sdX` 的动态命名方式可能会在插拔硬盘顺序不同的的时候出现混乱，采用软连接，如 uuid，其文件名不会随着插拔顺序等改变，只与硬盘设备本身有关。

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/lab-repo$ ls /dev/disk
by-diskseq  by-id  by-path  by-uuid
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/lab-repo$ ls /dev/disk/by-uuid
0a765a2d-bf06-4325-9a67-9f28a08eccf3  256f9195-dd3f-421a-9508-3cfb11901f88
```

运行 `lsblk` 会列出该系统拥有的所有块设备，从左往右分别是块设备的名称，设备的主/次设备号，`RM` 和 `RO` 分别表示是否可移除以及是否只读，`SIZE` 指的是盘容量的大小，`TYPE` 指的是块设备类型（`disk` 表示整个硬盘），`MOUNTPOINTS` 指的是该块设备的挂载点。

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/lab-repo$ lsblk
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 388.6M  1 disk
sdb   8:16   0   186M  1 disk
sdc   8:32   0     4G  0 disk [SWAP]
sdd   8:48   0     1T  0 disk /mnt/wslg/distro
```

5.3. tmpfs

Temporal File System，临时文件系统。

`.pid` 文件下存储的是程序自己的 PID，当程序启动时，会先检查该 `.pid` 文件中的指对应的进程是否还在运行，如果仍在运行则该进程直接退出，从而保证了程序只运行一个实例。

当多个进程竞争一个资源时，第一个创建并占用 `.lock` 文件的进程获得资源的访问权，后续进程发现 `.lock` 文件被锁则会阻塞或自旋等待，从而保证了资源只被一个程序访问。

在进程间通信时，服务端进程会绑定到该 `.socket` 并监听，客户端程序通过访问该文件与服务端建立通信。

因为 `tmpfs` 在内存中，因此上述的三个功能的运行速度很快，性能高；同时由于内存会断电后易失，因此可以保护其中存储的一些信息在断电后无法通过缓存等进行恢复；同时由于这些临时文件放在内存上，因此可以减少对磁盘读写的磨损，同时可以防止程序生成的数据让硬盘写满，造成系统崩溃，提升了系统整体的稳定性。

5.4. chroot 前的挂载

第一条指令是对 `/proc` 进行挂载，后面的参数表示禁止执行程序等，这个文件把内核的状态以及正在运行的进程等信息挂载到新的磁盘上，如果没有这些信息，我们将无法在新的根目录上执行 `ps` 等命令，以及无法进行某些如设置 root 密码等的操作。

第二条命令挂载了 `/sys` 目录，里面存储的是设备上硬件总线，电源管理等的配置信息，如果没有挂载，那么系统无法找到硬件设备的相关信息等，会导致 GRUB 引导程序无法正确安装（因为找不到 `/sys` 下的主板芯片接口），导致新系统无法被机器识别。

第三条命令挂载了 `/dev` 目录，里面放的是各种外设（如键盘，鼠标等）以及硬盘等的接口，如果没有这些接口那么操作系统就无法与用户进行交互，新系统安装时 GRUB 也无法向硬盘写入引导代码，导致无法正常安装。

6. 思考题

6.1. 为什么在 `crates/storage/lib.rs` 中声明了 `#![cfg_attr(not(test), no_std)]`，它有什么作用？哪些因素导致了 `kernel` 中进行单元测试是一个相对困难的事情？

声明 `#![cfg_attr(not(test), no_std)]` 相当于告诉了编译器如果当前不是在进行单元测试，就启用 `#![no_std]` 属性，它保证了在进行单元测试的时候可以连接 `std` 库，便于测试。因为我们进行单元测试的时候可能会需要用到 `println!` 等，如果要在我们的内核上进行测试需要初始化串口驱动等，其中也需要初始化文件系统，这就导致了我们进行单元测试就要整个 OS 进行 `make run`，且可能依赖于我们本身实现的单元，因此在 `kernel` 中如果不借用 `std` 中提供的各类方法，会相对困难。

6.2. 留意 `MbrTable` 的类型声明，为什么需要泛型参数 `T` 满足 `BlockDevice<B> + Clone`？为什么需要 `PhantomData<B>` 作为 `MbrTable` 的成员？在 `PartitionTable` trait 中，为什么需要 `Self: Sized` 约束？

因为 `MbrTable` 的成员函数 `.parse()` 中需要调用类型 `T` 的 `.read_block()` 成员函数，且 `.partition()` 成员函数需要对类型 `T` 进行 `clone`，因此需要类型 `T` 是一个块设备，具有 `BlockDevice<B>` trait，同时也要实现了 `Clone` trait 可以进行拷贝。其次，又因为 rust 要求结构体中声明的泛型类型需要在该结构体的字段中声明过，因此我们需要加入一个零大小类型 `PhantomData<B>` 变量来通过编译。最后，由于我们的 `PartitionTable` trait 中需要实现一个 `fn parse(inner: T) -> FsResult<Self>` 返回 `Self` 类型的函数，rust 要求

需要在编译器确定所有返回值的具体大小，因此需要加上 `Self: Sized` 约束，禁止实现 `fn parse(&dyn PartitionTable) -> Self` 这种返回值类型不确定的函数。

6.3. `AtaDrive` 为了实现 `MbrTable`，如何保证了自身可以实现 `Clone`？对于分离 `AtaBus` 和 `AtaDrive` 的实现，你认为这样的设计有什么好处?

`AtaDrive` 结构体下仅存储了总线编号，磁盘编号，磁盘的块数以及型号和序列号的字符串，这些类型都是具有 `Clone` trait 的，因此我们的 `AtaDrive` 自身可以实现 `Clone`，在访问的时候只需要通过这些信息在全局的 `BUSES` 数组上进行加锁访问即可。

我们在总线上进行读写的时候，我们的锁可以加在总线上，而不是直接写在全局或者某个具体的主设备或者从设备上，这样实现可以降低实现互斥的难度。其次，这样更符合我们实际抽象出来的模型，即总线和具体的块设备分离，可以同时对磁盘进行访问请求，但需要获取总线的锁才能传输数据。

6.4. 结合本次实验中的抽象和代码框架，简单解释和讨论如下写法的异同

`fn f<T: Foo>(f: T) -> usize` 和 `fn f(f: impl Foo) -> usize` 这两种写法本质上的效果是一样的，都是指静态分发，要求传入参数的类型 `T` 实现了 `Foo` 这一个 trait，底层实际上是对每个实现了 `Foo` trait 的类型都在编译器创建了一份独立的针对该类型的函数，会让编译生成的 ELF 文件偏大，好处是可以和普通函数一样运行，性能较高。

而 `fn f(f: &dyn Foo) -> usize` 是动态分发，要求传入的参数实现了 `Foo` trait，底层实现是通过虚函数表来确定实际调用的函数的，如下方代码

```
fn f(f: &dyn Foo) -> usize {
    f.g();
    // ...
}
```

在传入参数的时候，传入的 `f` 的指针有一部分是指向该类型对应的虚函数表，在运行时确定调用的成员函数 `g()` 的入口地址。这样的好处是生成的 ELF 文件较小，坏处是由于每次进行函数调用的时候都要查虚函数表，性能较低。

`struct S<T: Foo> { f: T }` 这样声明的结构体是静态的，要求 `T` 实现了 `Foo` trait，其内存分配是在编译器确定的，`f` 的具体大小会根据实际使用时类型 `T` 的大小确定。

而 `struct S { f: Box<dyn Foo> }` 是动态确定大小的，内部的 `Box` 表示该数据的内存空间是在运行是动态进行堆分配的，内部的 `f` 实际上是一个指向堆上数据的指针。

6.5. 文件系统硬链接和软链接的区别是什么？Windows 中的 “快捷方式” 和 Linux 中的软链接有什么异同?

Linux 文件系统中的每个文件指向一个 `inode` 号，记录了文件的权限，大小，修改时间以及在硬盘上的位置等。硬链接指的是在文件中创建一个新的文件名，但指向的是同一个 `inode`，

只有当所有指向该 `inode` 的硬链接删除后该文件数据才会删除，且由于不同分区的 `inode` 编码系统独立，因此不能跨分区建立硬链接，且不能对某个目录建立硬链接；而软链接是一个独立的新文件，有自己独立的 `inode` 号，指向的文件存储的信息是目标文件的路径，链接的时候根据路径寻找到对于的文件或者目录，此时链接指向的文件不会被软链接限制，可以直接删除，但此时软链接的路径就会找不到对应的文件。

Windows 中的“快捷方式”和软链接一样可以指向某个文件或者目录，且不会限制文件路径下文件的存在。但 Windows 下的“快捷方式”是由 shell 处理的，如果目标文件移动了，可以根据目标的 GUID 来找到目标文件。这个“快捷方式”文件实际上是 shell 的配置文件，我们不能在代码或者操作系统内核直接访问。而 Linux 软连接由于实际是一个路径，因此我们可以直接访问该文件，操作系统内核会直接重定向到目标文件。

6.6. 日志文件系统（如 NTFS）与传统的非日志文件系统（如 FAT）在设计和实现上有哪些不同？在系统异常崩溃后，它的恢复机制、恢复速度有什么区别？

非日志文件系统在写入一个文件的时候，会直接去修改文件分配表和实际的数据，这些操作是异步且非原子性的；而日志文件系统会在磁盘上建立一个专门的日志区，在修改文件数据之前会先把要执行的操作追加到日志区中，然后在开始执行修改。

在系统异常崩溃重启时，非日志文件系统会扫描整个磁盘，检查扇区和文件分配表是否正常，如果发现孤立簇（在修改 FAT 链之后没有把链头写入目录）或者出现交叉链接（一个簇出现在两条 FAT 链中），会把这个出现问题的簇切出来变成一个独立的文件，会导致文件的丢失，且恢复速度较慢。

而日志文件系统会检查日志区，如果我们的日志已经写入完成（写入了日志的结束符），则把日志中记录的内容 replay 重新执行一次；如果未写入完成（未写入结束符），则直接把日志的内容删掉。这样在系统异常奔溃后，只需要根据日志的内容进行操作即可，恢复速度较快，且只会在未完全写入日志的时候丢失数据，系统的稳定性较高。

7. 加分项

我们先补全 `io.rs` 中的 `read_all` 函数，它要做的就是根据文件大小不断扩充缓冲区直到可以读下整份 `elf` 文件为止，具体而言我们可以用类似 `std::vector` 那样每次成倍扩展的方式扩充缓冲区。（`cat` 由于不需要保存整个文件的内容到缓冲区而只需要输出，因此采用了内存开销更小的边 `read` 边输出的方式）

然后我们更改一下 `spawn` 的实现，使用 `open_file` 和 `read_all` 函数来获取文件操作的句柄，以及读入整个 `elf` 文件，然后解析 `elf` 文件之后和之前的 `spawn` 一样交给 `elf_spawn` 即可。这部分代码请见 `proc/mod.rs`。

然后我们更改一下 `sh` 中的内容，在运行用户程序的时候自动加入 `/APP/` 前缀，然后调用当前的新 `spawn` 来运行，由于我们只支持短文件名，因此部分用户程序的名称有所变化，可以使用 `ls` 查看，如下

```
[+] App list:
  App Name        | Size       | Entry Point
  ----------------+------------+-------------------
  counter         |    38992 B | 0x0000111100001470
  mq              |    38904 B | 0x00001111000017e0
  fork_test       |    35456 B | 0x00001111000011d0
  sleep           |    34416 B | 0x00001111000010a0
  hello           |    34320 B | 0x0000111100001020
  dinner          |    34480 B | 0x00001111000014f0
  fish            |    38792 B | 0x0000111100001320
  factorial       |    34872 B | 0x0000111100001490
  sh              |    39664 B | 0x0000111100001960

[>] ls /APP/
Name                    Size Created
--------------- ---------- --------------------
./                         - 2026-05-09 13:17:14
../                        - 2026-05-17 20:00:58
COUNTER              38.1 K 2026-05-17 20:23:40
MQ                   38.0 K 2026-05-17 20:23:40
FORK_T~1             34.6 K 2026-05-17 20:23:40
SLEEP                33.6 K 2026-05-17 20:23:40
HELLO                33.5 K 2026-05-17 20:23:40
DINNER               33.7 K 2026-05-17 20:23:40
FISH                 37.9 K 2026-05-17 20:23:40
FACTOR~1             34.1 K 2026-05-17 20:23:40
SH                   38.7 K 2026-05-17 20:23:40
```

这是由于 QEMU 自带了虚拟化 FAT 功能，会在启动的时候把带 `LFN` 属性的长名称截断 + `~` + 数字生成的短名称都放入了磁盘扇区，因此我们根据 standard 8.3 实现的 FAT16 文件系统可以正常处理以其生成的短名称来这种长名称文件。

于是我们就可以根据目前的新名称，运行对应的用户程序了！

```
FISH                37.9 K 2026-05-17 20:23:40
FACTOR~1            34.1 K 2026-05-17 20:23:40
SH                  38.7 K 2026-05-17 20:23:40
[>] factor~1
Input n: 999999
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  1 | #  0 | kernel       | 3718011 | Ready | 2097152 B (512 Pages)
 # 23 | #  2 | /app/factor~1 |   10387 | Running | 16027648 B (3913 Pages)
 #  2 | #  1 | /app/sh      | 3697668 | Blocked | 32768 B (8 Pages)
Queue  : [1]
CPUs   : [0: 23]
The factorial of 999999 under modulo 1000000007 is 128233642.
Process [23] exited with code: 0
```